10-2026

code URJC

# Fundamentos de la Web

## Bloque III: Tecnologías de servidor web

# Tema 3.2: Lenguaje JavaScript

Universidad Rey Juan Carlos

Micael Gallego, Francisco Gortázar, Michel Maes, Óscar Soto e Iván Chicano

code
URJC

©2024

Micael Gallego, Francisco Gortázar, Michel Maes, Óscar Soto, Iván Chicano

Algunos derechos reservados

Este documento se distribuye bajo la licencia
"Atribución-CompartirIgual 4.0 Internacional"
de Creative Comons Disponible en
https://creativecommons.org/licenses/by-sa/4.0/deed.es

# Lenguaje JavaScript

- Características del lenguaje
- Sintaxis básica
- Arrays
- Sentencias de control de flujo
- Funciones
- Orientación a Objetos
- Excepciones



# Lenguaje JavaScript

- **Características del lenguaje**
- Sintaxis básica
- Arrays
- Sentencias de control de flujo
- Funciones
- Orientación a Objetos
- Excepciones



# Características del lenguaje JS

- Vamos a estudiar las características **más usadas** actualmente para desarrollar aplicaciones
- Hay ciertas características que ya **no se aconsejan y no veremos** (aunque te las puedes encontrar si ves código antiguo)
- Como estáis aprendiendo **Java**, haremos una comparativa entre ambos

# Características del lenguaje JS

code URJC

- **JS es un lenguaje de script (Java es compilado)**
  - No existe compilador
  - El navegador carga el código, lo analiza y lo ejecuta
  - El navegador indica tanto **errores de sintaxis** como errores de **ejecución**
- **JS tipado dinámico (Java tiene estático)**
  - Al declarar una variable no se indica su tipo
  - A lo largo de la ejecución del programa una misma variable puede tener **valores de diferentes tipos**

# Características del lenguaje JS

- **Imperativo y estructurado (como Java)**
  - Se declaran **variables**
  - Se ejecutan las sentencias **en orden**
  - Dispone de **sentencias de control** de flujo de ejecución (if, while, for…)
  - La **sintaxis** imperativa/estructurada es muy parecida a Java y C

# Características del lenguaje JS

code URJC

- **Orientado a objetos**
  - Todos los **valores son objetos** (no como en Java, que existen tipos primitivos)
  - Existe **recolector de basura** para liberar la memoria de los objetos que no se utilizan (como en Java)
  - La **orientación a objetos** puede basarse en clases o en prototipos (Java se basa en clases)
  - En tiempo de ejecución se pueden **crear objetos**, cambiar el valor de los **atributos** e invocar a **métodos** (como en Java)
  - En tiempo de ejecución se pueden **añadir y borrar atributos y métodos** (en Java no se puede)

# Características del lenguaje JS

code URJC

- **Funciones**
  - Aunque sea orientado a objetos, también permite declarar **funciones independientes** (en Java todas las funciones son métodos de clases)
  - Las funciones se pueden **declarar en cualquier sitio**, asignarse a variables y pasarse como parámetro
  - Existen **funciones anónimas** (como las **expresiones lambda** de Java)
  - En JavaScript se puede implementar código inspirado en el **paradigma funcional**

# Lenguaje JavaScript

- Características del lenguaje
- **Sintaxis básica**
- Arrays
- Sentencias de control de flujo
- Funciones
- Orientación a Objetos
- Excepciones



# Sintaxis básica

- **Comentarios (como en Java)**
  - Una línea //
  - Multilínea /* ..... */
- **Delimitadores (como en Java)**
  - De bloque { }
  - De sentencia ; (opcionales)
- **Palabras reservadas**

```
abstract boolean break byte case catch char class const continue debugger default
delete do double else enum export extends false final finally float for function goto
if implements import in instanceof int interface long native new null package private
protected public return short static super switch synchronized this throw throws
transient true try typeof var volatile void while with
```

# Sintaxis básica

- **Variables**
  - Es un lenguaje dinámico (con tipado **dinámico**)
  - Las variables se tienen que declarar pero **no se indica el tipo**

```
// Variable age will have value 34
let age = 34;
let found = false;
```

  - La variable está disponible en el bloque en el que se declara (como en Java)

# Sintaxis básica

- **Variables**
  - Una misma variable puede tener un valor numérico en un punto de ejecución y una cadena de caracteres tiempo después

```
let id;
...
id = 34;
...
id = "XDFS";
```

# Sintaxis básica

- **Variables**

**ejem1-undefined**

  - Si las variables **no se inicializan** tienen el valor **undefined** (en vez de **0** o **null** como en Java)

```
let hello;
console.log("Hello: " + hello);
```



```
Hello: undefined
```

  - Si una **variable se usa sin haberse declarado** con let (por una equivocación del desarrollador)
    - Si se intenta **leer su valor** salta un **error** (que finaliza la ejecución del programa)
    - Si se **asigna un valor**, se crea un nuevo atributo en el objeto global y **no dan error las siguientes lecturas**

# Sintaxis básica

- **Algunos tipos de datos básicos**
  - Son objetos (no existe distinción como en Java entre objetos y tipos primitivos)
  - **Number:** Números enteros y reales de cualquier precisión
  - **Boolean:** true o false (como en Java)

# Sintaxis básica

- **Algunos tipos de datos básicos**
  - **String**
    - Cadenas de caracteres
    - Comillas simples o dobles
    - No pueden modificarse. Inmutables (como en Java)
    - Concatenación con + (como en Java)
    - Como no hay tipo caracter, se usa un string con un único caracter

# Sintaxis básica

- **Algunos tipos de datos básicos**
  - **Template strings**
    - Comillas *backtick* `
    - Crean caracteres con valores de variables
    - Permiten strings multilínea

```
let name = 'Internal Error';
let statusCode = 500;
let message = 'Fatal error';

console.log(`InternalError: ${this.name} (${this.statusCode}): ${this.message}`);
console.log(
  'InternalError: '+ this.name + '(' + this.statusCode +'):' + this.message);
```

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Template_literals

# Sintaxis básica

- **Operadores en expresiones**
  - **Similares a Java**
    - Aritméticos: + - * / % (la división es siempre real)
    - Comparación números: < > <= >=
    - Lógicos: && || !
    - Comparativo: ? :
    - Incremento/Decremento: ++ --
    - Asignación: = += -= *= /= %=

# Sintaxis básica

- **Operadores en expresiones**

ejem2-comparison

  - **Comparación (diferente a Java)**
    - Igual: `===` Distinto: `!==`
    - En **strings** se comporta como el equals(…) en Java (**mismos caracteres**)
    - En **arrays** se comporta como == en Java (**mismo objeto**)

```
let uniName = "URJC";
let otherName = "UR" + "JC";
console.log(uniName === otherName); // true
```

# Lenguaje JavaScript

code URJC

- Características del lenguaje
- Sintaxis básica
- **Arrays**
- Sentencias de control de flujo
- Funciones
- Orientación a Objetos
- Excepciones



# Arrays

code URJC

- **Los arrays de JavaScript son parecidos a los de Java**
  - El acceso para lectura o escritura es con **[ ]**
  - Se inicializan con **[ ]** (en Java con **{ }**)
  - Tienen la propiedad **length**
  - Empiezan por **cero**

**ejem3-arrays**

```
let numbers = [3, 4, 4, 2];
console.log(numbers[0]); // 3
console.log(numbers.length); // 4
```

# Arrays

code URJC

- **Los arrays de JavaScript son parecidos a los de Java**
  - La **asignación** de una variable con un array a otra variable, no copia. Ambas variables apuntan al **mismo objeto array**
  - El operador **===** compara si son el **mismo objeto** (no que tengan el contenido igual)

**ejem4-array-comparison**

```
let nums = [1, 2, 3, 4];
let numsAux = nums;
numsAux[0] = -1;
console.log(nums[0] === -1); // true
console.log(numsAux === nums); // true
```

# Arrays

- **Los arrays de JavaScript son parecidos a los de Java**
  - Los **arrays de varias dimensiones son arrays de arrays**. Hay que crear de forma explícita los niveles.
  - Se pueden recorrer de forma similar

```
let numbers = [
    [1, 2],
    [3, 4]
];

console.log(numbers[0][0] + ',' +
            numbers[0][1] + ',' +
            numbers[1][0] + ',' +
            numbers[1][1]);
```

**ejem5-array-2d**



```
1,2,3,4
```

# Arrays

- **Pero se diferencian en algunos detalles**
  - Los arrays literales con **[]** en vez de **{}** y sin **new**

```
let empty = [];
let numbers = ['zero','one','two','three'];
```

  - Un array en JS puede tener en cada posición valores de diferentes tipos mezclados (tipado dinámico)

```
let someData = ['zero', 1, false, undefined];
```

# Arrays

code URJC

- **Errores de acceso en arrays**
  - La lectura de un elemento fuera de los límites devuelve **undefined**
    - En Java sería un `ArrayIndexOutOfBoundsException`
  - La escritura de un elemento fuera de los límites del array se permite, haciendo más grande el array y rellenando con valores **undefined** los huecos

# Arrays

code URJC

**ejem6-array-empty-positions**

```
let numbers = [3, 4];
console.log(numbers[2]);
numbers[4] = 5;
console.log(numbers[4]);

console.log(numbers[0] + ',' +
            numbers[1] + ',' +
            numbers[2] + ',' +
            numbers[3] + ',' +
            numbers[4] + ',');
```



```
undefined
5
3,4,undefined,undefined,5
```

# Arrays

code URJC

- **Errores de acceso en arrays**

ejem7-array-undefined

  - Si una variable tiene el valor **undefined** y se intenta acceder a un elemento como si apuntara a un array, genera un error "**Uncaught TypeError: Cannot read properties of undefined (reading '5')**"

```
let numbers;
console.log(numbers[5]);
```

  - En Java sería un `NullPointerException`

# Arrays

- **Modificación del array**
  - Se pueden establecer elementos en posiciones no existentes y el array crece dinámicamente.
  - El método **push(…)** añade un elemento al final del array (como el método **add(…)** del **ArrayList** en Java)
  - Modificando la propiedad **length** se puede cambiar el tamaño del array (si se amplía, se rellena con undefined) (en Java es de sólo lectura)

# Arrays

- **Modificación del array**
  - El **operador delete** borra un elemento dejando el hueco con valor **undefined**

```
delete numbers[2];
```

  - Para borrar y no dejar el hueco se usa el método **splice(…)** indicando el índice desde el que hay que borrar y el número de elementos que borrar

```
numbers.splice(2, 1);
```

# Lenguaje JavaScript

- Características del lenguaje
- Sintaxis básica
- Arrays
- **Sentencias de control de flujo**
- Funciones
- Orientación a Objetos
- Excepciones



# Sentencias de control de flujo

code URJC

- **Sentencia if**
  - Sintaxis como en Java
  - No es obligatorio que la expresión devuelva un boolean
  - Se consideran como valores falsos: `false`, `null`, `undefined`, “” (cadena vacía), `0`, `NaN`

# Sentencias de control de flujo

code URJC

ejem8-if

```
let grade = 7;
...
if(grade < 5) {
   console.log('Failed');
} else {
   console.log('Passed');
}
```

```
let username = '';

if(!username){
   username = 'No name';
}
```

# Sentencias de control de flujo

code URJC

ejem9-switch-for

- **Sentencias switch, while, do while, for**
  - Sintaxis y semántica como en Java y C

```
let animal = 'Giraffe';

switch (animal) {
   case 'Cow':
   case 'Giraffe':
   case 'Pig':
     console.log('Goes on the Ark');
     break;
   case 'Dinosaur':
   default:
   console.log('Doesn’t go');
}
```

```
let array = [3,4,5,6,4,5]

for (let i=0; i<array.length; i++) {
    console.log(array[i]);
}
```

# Lenguaje JavaScript

code URJC

- Características del lenguaje
- Sintaxis básica
- Arrays
- Sentencias de control de flujo
- **Funciones**
- Orientación a Objetos
- Excepciones



# Funciones

- En **JavaScript** las funciones se pueden declarar de forma independiente, sin estar vinculadas a una clase
- Se pueden declarar con nombre

**ejem10-functions**

```
function printParam(param){
   console.log(param);
}

printParam(4);
```

# Funciones

- Se pueden declarar **sin nombre** (**anónimas**) y asignarse a una **variable** o **parámetro**
- Se invocan usando el **nombre de la variable** o parámetro seguido de los **paréntesis**

**ejem11-anonymous-functions**

```
let printParam = function(param){
   console.log(param);
}

printParam(4);
```

# Funciones

- **Sentencia return**
  - Cuando se ejecuta fuerza la terminación de una función (como en Java)
  - Puede tener un valor asociado que será devuelto por la función en la que se ejecute

**ejem12-return**

```
function myFunction(a, b) {
  return a * b;
}

console.log(myFunction(4, 3));
```

# Funciones

- Las funciones se pueden pasar como **parámetro** a otras funciones

```
function showItems(array, hasToShow) {
    for(let i=0; i<array.length; i++){
        if(hasToShow(array[i])){
            console.log(array[i]);
        }
    }
}

function isEven(number){
    return number % 2 == 0;
}

showItems([1,2,3,4,5,6], isEven);
```



```
2
4
6
```

**ejem13-functions-in-parameter**

# Funciones

- Se pueden declarar funciones en el cuerpo de otras funciones

**ejem14-functions-in-functions**

```
function showNonZeros(numbers) {

        let isNonZero = function (num) {
                return num != 0;
        }

        showItems(numbers, isNonZero);
}
```

# Funciones

- Se pueden declarar funciones en la llamada a otras funciones

**ejem15-functions-declaration-in-parameter**

```
function showNonZeros(numbers) {

   let isNonZero = function (num) {
          return num != 0;
   }

   showItems(numbers, isNonZero);
}
```



```
function showNonZeros(numbers) {

    showItems(numbers, function (num) {
        return num != 0;
    });
}
```

# Ejercicio 2

code URJC

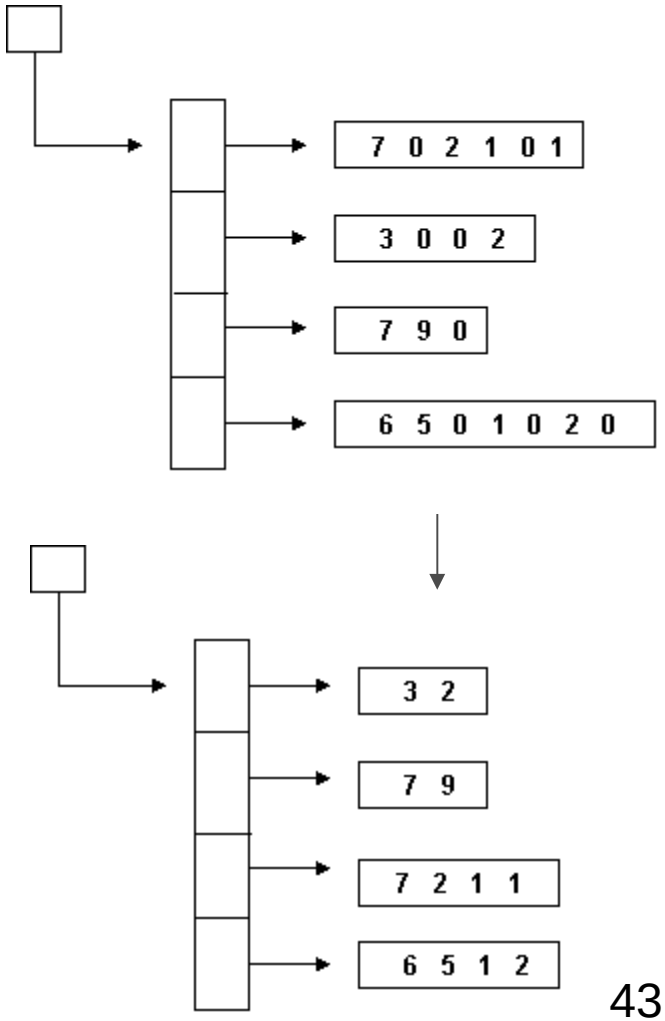
**ejer2-functions-even-odd-2d-array**

- Crear una función que **reciba un array como parámetro** y devuelva un array de dos elementos
- El primer elemento apuntará a un **array con los números pares** del array que se pasa como parámetro
- El segundo elemento apuntará a un **array con los números impares** del array que se pasa como parámetro
- Para probar la función se implementarán varias llamadas con **diferentes arrays** y el resultado se mostrará en la **consola**

# Ejercicio 3

**ejer3-functions-remove-zeroes-and-sort**

- Crear una función que **reciba un array bidimensional, le quite los ceros y ordene las filas de menor a mayor longitud**
- Para probar la función se implementarán varias llamadas con **diferentes arrays** y el resultado se mostrará en la **consola**



# Ejercicio 3

- Para ordenar los arrays por tamaño se puede usar el **algoritmo de la burbuja** (*bubble sort*)
- Ejemplo: Ordenación de un array de números (tiene que adaptarse para ordenar arrays por su tamaño)

```
function sortNumbers(numbers) {
   for (let i = 0; i < numbers.length; i++) {
      for (let j = 0; j < numbers.length - n - 1; j++) {
         if (numbers[j] > numbers[j + 1]) {
            let temp = numbers[j];
            numbers[j] = numbers[j + 1];
            numbers[j + 1] = temp;
         }
      }
   }

   return numbers;
}
```

# Funciones

code URJC

- **Parámetros de una función**
  - Si se pasan menos parámetros de los que están en la cabecera, los que faltan toman el valor **undefined (No hay error)**

**ejem16-functions-undefined-param**

```
function printParam(param){
   console.log(param); //undefined
}

printParam();
```

# Funciones

- Como pasar **menos parámetros no da error**, se puede usar para implementar parámetros **opcionales**

**ejem17-functions-optional-params**

```
function arrayToString(array, separator){
    if(!separator){
            separator = ",";
    }
    let result = "";
    for(let i=0; i<array.length; i++){
            result += array[i] + separator;
    }
    return result;
}

console.log(arrayToString([1,2,3,4]));
console.log(arrayToString([1,2,3,4],"-"));
```

```
1,2,3,4,
1-2-3-4-
```

# Funciones

- **Parámetros de una función**
  - Si se pasan más parámetros de los que están en la cabecera, los que sobran se **ignoran (No hay error)**

**ejem18-functions-excess-params**

```
function printParam(param){
   console.log(param); //5
}

printParam(5, 7);
```

# Funciones

- **Información accesible desde la función**
  - Parámetros y variables declaradas en la función

**ejem19-functions-local-scope**

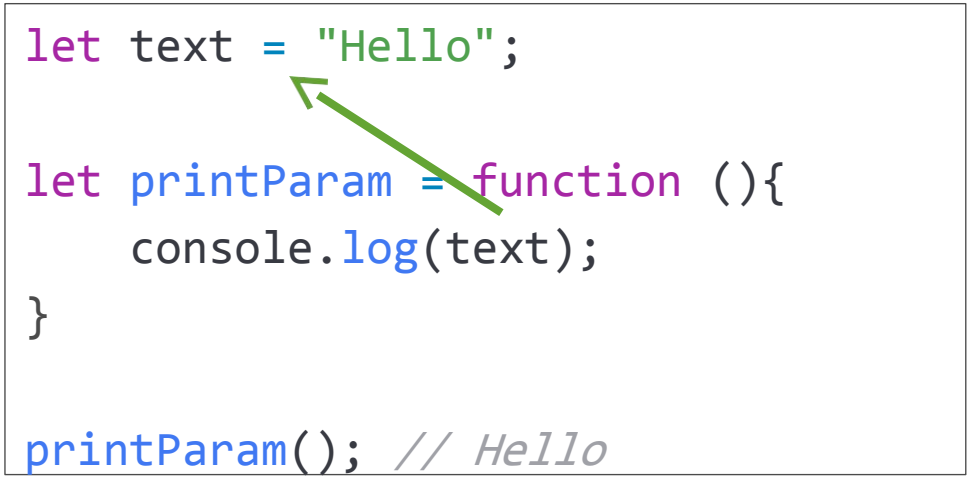
```
let printParam = function (param){
    let local = 0;
    console.log("param:"+param+" local:"+local);
}

printParam(4); // param:4 local:0
```

# Funciones

- **Información accesible desde la función**
  - Variables accesibles en el punto en que se declara la función (ámbito léxico)

**ejem20-functions-same-scope**



# Funciones

- **Información accesible desde la función**
  - Cuando se referencia a una variable no sólo se accede a su valor, se accede a la **propia variable en sí**
  - Si se **cambia el valor** de la variable, la **función podrá leer** el nuevo valor
  - En Java las variables a las que accede una expresión lambda **no pueden cambiar de valor**

# Funciones

- **Información accesible desde la función**
  - El conjunto de variables a las que tiene acceso la función se **llama cerradura o cierre (*closure*)**

**ejem21-functions-closure**

```
let text = 'Hello'

function printParam(){
   console.log(text)
}

printParam(); // Hello
text = 'Goodbye'
printParam(); // Goodbye
```

# Funciones flecha (arrow function)

- Una forma más compacta de escribir funciones anónimas

```
let f = function (v){
    return v + 1;
}
```



```
let f = v => v + 1;
```

```
var f = function (p1,p2){
    console.log('Hello');
    console.log('Goodbye');
    return 3;
}
```



```
var f = (p1,p2) =>{
    console.log('Hello');
    console.log('Goodbye');
    return 3;
}
```

# Funciones flecha (arrow function)

- Muy usado cuando la función se declara en la lista de parámetros de otra función

**ejem22-functions-arrow-function**

```
function showNonZeros(numbers) {
    showItems(numbers, function (num) {
        return num != 0;
    });
}
```

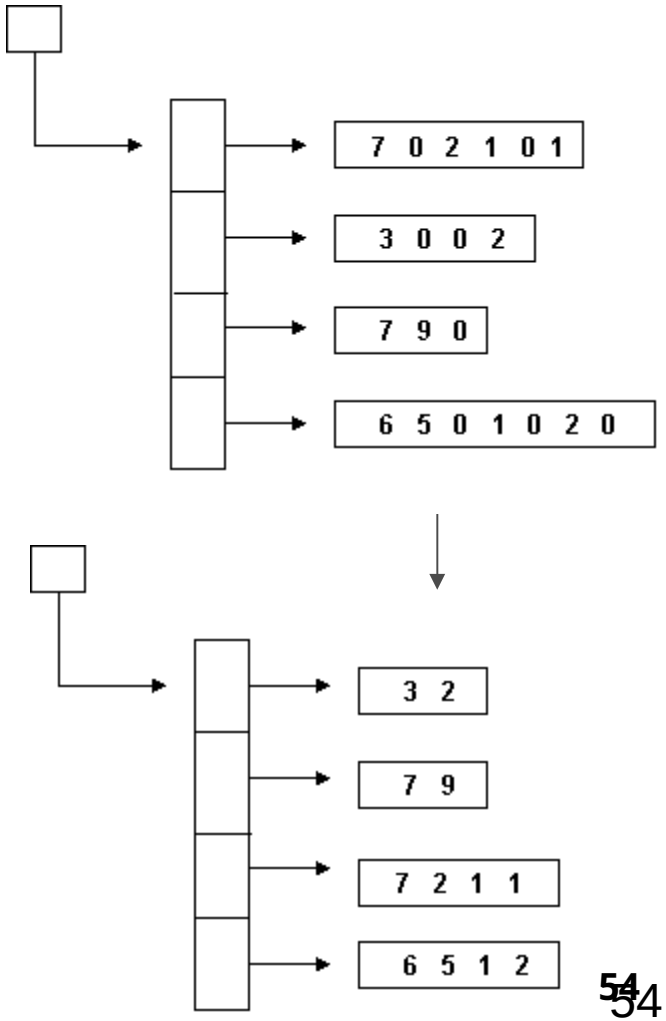


```
function showNonZeros(numbers) {
   showItems(numbers, num => num != 0);
}
```

# Ejercicio 4

**ejer4-functions-remove-nums-and-sort**

- Haz que el ejercicio 3 sea configurable
- En vez de eliminar los ceros de los arrays, que elimine los **números que decida el usuario** pasando una función como parámetro
- Usa arrow functions en las pruebas que llaman a `removeNumsAndSort(...)`



# Lenguaje JavaScript

code URJC

- Características del lenguaje
- Sintaxis básica
- Arrays
- Sentencias de control de flujo
- Funciones
- **Orientación a Objetos**
- Excepciones



# Orientación a Objetos

code URJC

- Hasta ahora hemos visto que **JavaScript** es un lenguaje bastante familiar en cuanto a **sintaxis** y **características:**
  - Imperativo y estructurado
  - Tipado dinámico
  - Algunos tipos básico: Number, Boolean y String
  - Arrays dinámicos
  - Recolector de basura
  - Funciones

# Orientación a Objetos

code URJC

- La orientación a objetos en **JavaScript** a primera vista parece **similar** a la de **Java**, pero en esencia es **muy diferente**
- La gran **mayoría de los lenguajes** de programación implementan la Orientación a Objetos con **clases** (Java, C#, C++, Python, Ruby…)
- En JavaScript se añadió el soporte de **clases** en versiones recientes. Hasta ese momento se usaban los **prototipos** (que no veremos aquí)

# Orientación a Objetos

code URJC

- **Creación de objetos**
  - En cualquier momento se puede **crear un objeto, definir sus atributos** y asignarles **valor**

**ejem23-oop-properties**

```
let employee = {
    name: "Pepe",
    salary: 700
}
```

# Orientación a Objetos

code URJC

- Atributos de un objeto

ejem23-oop-properties

```
let employee = {
   name: "Pepe",
   salary: 700
}

console.log("S:"+ employee.salary);

employee.salary = 800;

employee.phone = "663232539";

console.log("T:"+ employee.phone);
```

# Orientación a Objetos

- **Métodos en un objeto**
  - Los objetos pueden tener **métodos**
  - **Los métodos son en realidad** funciones asignadas a una propiedad

```
let employee = {
    name: "Pepe",
    salary: 700,
    toString: function () {
      return "N:" + this.name + " S:" + this.salary;
    }
}
```

Para acceder a los atributos del objeto es necesario usar **this** (en Java es opcional)

# Orientación a Objetos

code URJC

- **Métodos en un objeto**
  - Los métodos se **invocan** con la notación punto (como en Java)

```
//Returns 'N:Pepe, S:700'
let text = employee.toString();
```

  - Se pueden **añadir métodos** a un objeto en cualquier momento

```
empleado.getCategory = function(){
    return this.salary > 800 ? 'Superior':'Normal';
}
```

# Orientación a Objetos

code URJC

- **Métodos en un objeto**

**ejem24-oop-methods**

```
let employee = {
   name: 'Pepe',
   salary: 700,
   toString: function () {
      return 'N:' + this.name + ' S:' + this.salary;
   }
}

// Shows N:Pepe S:700
console.log(employee.toString());

employee.getCategory = function () {
   return this.salary > 800 ? 'Superior' : 'Normal';
}

console.log('C:' + employee.getCategory());
```

# Orientación a Objetos

code URJC

- Como se puede ver, los **objetos son muy flexibles** porque **no necesitan** un **molde** (una clase)
- Esta forma de trabajar es muy útil cuando **sólo hay un objeto** con una estructura determinada (***singleton***)
- Se usa bastante en JavaScript para crear **objetos sin métodos** para manipular información

# Ejercicio 5

code URJC

**ejer5-oop-rectangle-area-perimeter**

- Se quiere implementar un programa en JavaScript que calcule el área y perímetro de un rectángulo
- El rectángulo se representará como un objeto con los atributos:
  - color
  - height
  - width
- Y métodos:
  - area(): height * width
  - perimeter(): 2 * height + 2 * width

# Orientación a Objetos

code URJC

- Crear varios objetos con los mismos atributos y métodos

**ejem25-oop-create-new-objects**

```
let employee = newEmployee("Pepe",700);

//Returns 'N:Pepe, S:700'
console.log(employee.toString());

//Returns 700
console.log(employee.salary);
```

# Orientación a Objetos

code URJC

- Crear varios objetos con los mismos atributos y métodos

**ejem25-oop-create-new-objects**

```
function newEmployee(name, salary) {

    let employee = {
        name: name,
        salary: salary,
        toString: function () {
            return "N:" + this.name + " S:" + this.salary;
        }
    };

     return employee;
}
```

# Orientación a Objetos

code URJC

- **Objetos, null y undefined**
  - En JavaScript también se puede usar **`null`** como un valor válido para las variables
  - Las variables que no están inicializadas tienen el valor **`undefined`**
  - Usar una variable sólo si apunta a un objeto

```
let obj = null;

if(obj){
   obj.doSomething();
}
```

Devuelve **`true`** si `obj` no es **`null`** ni es **`undefined`**

# Ejercicio 5b

code URJC

**ejer5b-oop-rectangle-array-analysis**

- Se quiere implementar un programa en JavaScript que permita analizar un array de rectángulos
- De cada rectángulo se debe conocer:
  - Color (color)
  - Alto (height)
  - Ancho (width)
  - Área (area)
  - Perímetro (perimeter)

# Ejercicio 5b

code URJC

**ejer5b-oop-rectangle-array-analysis**

- Los **análisis** serán:
  - Suma total de áreas de los rectángulos
  - Suma total de perímetros de los rectángulos
  - Área media
  - Perímetro medio
- Prueba del **correcto funcionamiento**:
  - Construir un array con diferentes rectángulos
  - Ejecutar todos los análisis
  - Muestra el resultado en la consola

# Orientación a Objetos con clases

code URJC

- JavaScript también permite implementar **clases** (similares a las de Java)
- Se pueden crear **objetos** como **instancia de una clase** (eso hace que tenga sus métodos y atributos)
- Al igual que en Java se permite:
  - Constructor y métodos
  - Herencia de clases
  - Métodos/Atributos estáticos

# Orientación a Objetos con clases

code URJC

## Clase en JavaScript

```
class Employee {

   constructor(name, salary) {
      this.name = name;
      this.salary = salary;
   }

   getName() {
      return this.name;
   }

   toString() {
      return "N:" + this.name +
         ", S:" + this.salary;
   }
}
```

## Clase en Java

```
public class Employee {

   private String name;
   private double salary;

   public Employee(String name, double salary) {
      this.name = name;
      this.name = salary;
   }

   public String getName() {
      return name;
   }

   public String toString() {
      return "N:" + name + ", S:" + salary;
   }
}
```

# Orientación a Objetos con clases

code URJC

**ejem26-oop-classes**

```
class Shape {

   constructor(id, x, y) {
      this.id = id;
      this.move(x, y);
   }

   move(x, y) {
     this.x = x;
     this.y = y;
   }

   toString() {
      return 'Shape(' + this.id + ')';
   }
}
```

# Orientación a Objetos con clases

code URJC

**ejem26-oop-classes**

```
class Rectangle extends Shape {

   constructor(id, x, y, width, height) {
      super(id, x, y);
      this.width = width;
      this.height = height;
   }

   toString() {
      return "Rectangle > " + super.toString();
   }
}
```

```
let rectA = new Rectangle('A', 3, 5, 10, 20);
```

# Orientación a Objetos con clases

code URJC

**ejem26-oop-classes**

```
class Rectangle extends Shape {


    …
    static defaultRectangle() {
        return new Rectangle("default", 0, 0, 100, 100);
    }
}

let defRectangle = Rectangle.defaultRectangle();
```

# Orientación a Objetos con clases

code URJC

## Métodos y atributos privados

```
class ClassWithPrivate {
    #privateField;
    #privateFieldWithInitializer = 42;
    #privateMethod() {
        this.#privateField = "default";
    }

    static #privateStaticField;
    static #privateStaticFieldWithInitializer = 42;

    static #privateStaticMethod() {
        // …
    }
}
```

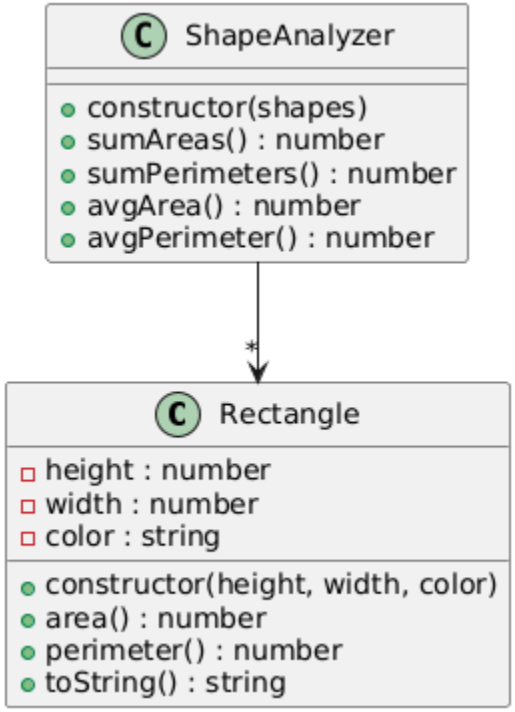
# Orientación a Objetos con clases

code URJC

- En JavaScript **no existe** algo equivalente a los **interfaces de Java**
- No son necesarios con **tipado dinámico**

# Ejercicio 6

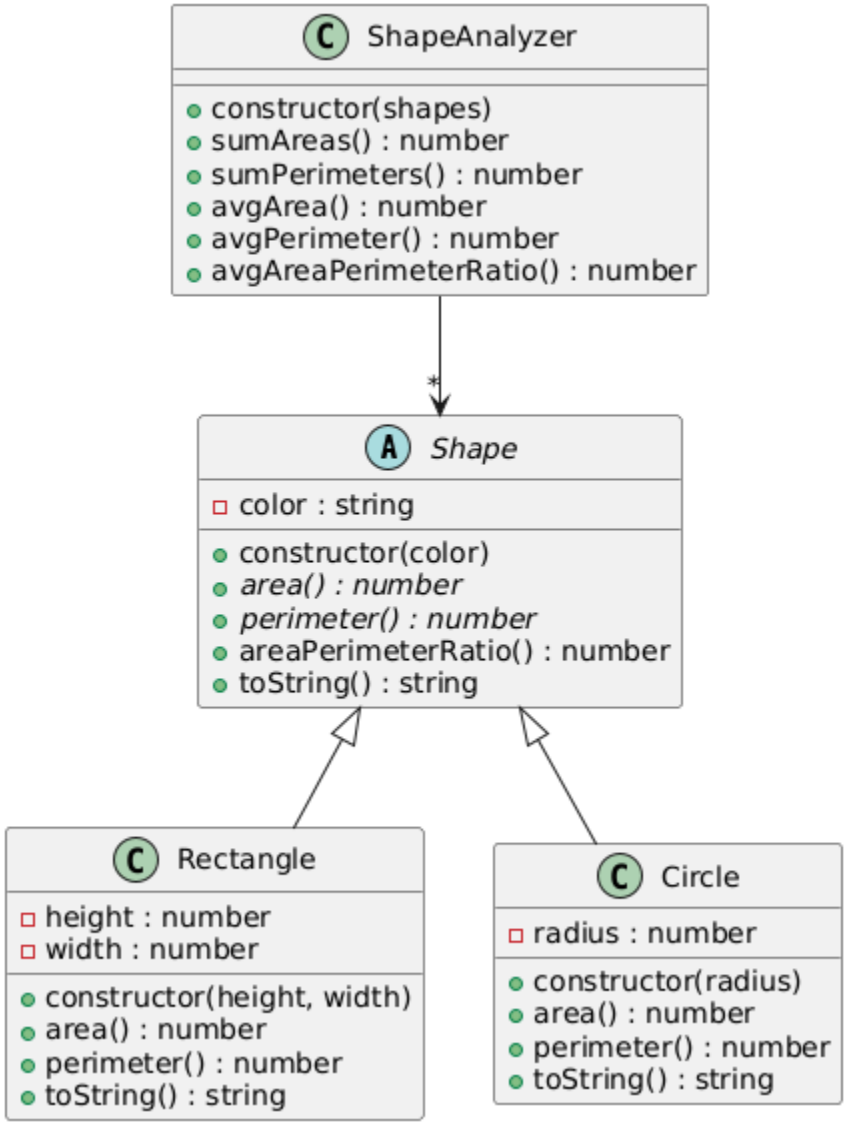
- Transforma el **Ejercicio 5** para que use **clases**
- Agrupa las **funciones de análisis** en la clase **ShapeAnalyzer**



# Ejercicio 7

- Amplía el **Ejercicio 6** con las siguientes funcionalidades
  - Añadir el **círculo** como nuevo tipo de figura
    - Área: Math.PI * radio * radio
    - Perímetro: 2 * Math.PI * radio
  - Añadir un **nuevo análisis**
    - Por cada figura saber el **ratio area/perímetro**
    - La **media del ratio área/perímetro** para todas las figuras

# Ejercicio 7


ShapeAnalyzer
constructor(shapes)
sumAreas() : number
sumPerimeters() : number
avgArea() : number
avgPerimeter() : number
avgAreaPerimeterRatio() : number
*
Shape
color : string
constructor(color)
area() : number
perimeter() : number
areaPerimeterRatio() : number
toString() : string
Rectangle
height : number
width : number
constructor(height, width)
area() : number
perimeter() : number
toString() : string
Circle
radius : number
constructor(radius)
area() : number
perimeter() : number
toString() : string

# Lenguaje JavaScript

- Características del lenguaje
- Sintaxis básica
- Arrays
- Sentencias de control de flujo
- Funciones
- Orientación a Objetos
- **Excepciones**



# Excepciones

- Funcionan igual que en **Java**
- Existe un bloque con **try catch finally**
- El operador **throw** eleva la excepción
- A diferencia de Java, se puede lanzar cualquier objeto como excepción, aunque lo recomendable es elevar un objeto con toda la información posible.

# Excepciones

code URJC

**ejem27-errors-try-catch**

```
try {
    if (error) {
        throw "An error";
    }
    return true;
} catch (e) {
    console.error(e);
    return false;
} finally {
    //do cleanup
}
```

# Excepciones

code URJC

**ejem28-errors-inheritance**

```
class BaseError extends Error {
   constructor(name, statusCode, isOperational, description) {
      super(description);

      this.name = name;
      this.statusCode = statusCode;
      this.isOperational = isOperational;
      //Sólo disponible en Node.js, no en un navegador
      //Error.captureStackTrace(this);
   }

   toString() {
      return `BaseError: ${this.message}`;
   }
}
```

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error

https://nodejs.org/api/errors.html#class-error

# Excepciones

code URJC

**ejem28-errors-inheritance**

```
class InternalError extends BaseError {
   constructor(description) {
      super('Internal Error', 500, true, description);
  }

   toString() {
      return `InternalError: ${this.name} (${this.statusCode}): ${this.message}`;
   }
}
```

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error

https://nodejs.org/api/errors.html#class-error

# Excepciones

code URJC

**ejem28-errors-inheritance**

```
try {
    throw new InternalError('Internal Error 500');
} catch (e) {
    console.log(e.toString());
    console.error(e);
}
```

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Error

https://nodejs.org/api/errors.html#class-error

# Ejercicio 8

- Amplía el **Ejercicio 7** implementando manejo de errores en los casos en los que puede haber operaciones no válidas.
  - Ejemplos:
    - No se debería poder instanciar la clase Shape.
    - No se debería poder llamar a los métodos area y perimetro de Shape, solo de sus hijos.
    - No se pueden calcular el área o el perímetro de una figura si alguna de las dimensiones de una figura son menor o igual que cero o un valor no numérico (isNaN()).
    - Otros casos que se te ocurran.